Parallel Programming Assignment Report

Students Information

Jannah Ayman - CS 1 - General
Rawan Sotohy - CS 2 - Genreal
Sohila Ahmed Usama - CS 2 - Genreal
Hana Salah - CS 4 - Genreal
Mayar Mohamed Ahmed - CS 4 - General
Shrouk Mohsen - CS – Civil

Repo Link

Problem 1

Problem Title: Serial PI

Lecture Number: Lec 2, Unit 2, Disc 2

Problem Description

Calculate the value of Pi using numerical integration on a single thread to serve as our baseline for comparison.

Parallelization Approach

None yet

Solution Explanation

Loop from 0 to N steps, compute the midpoint x of each rectangle, evaluate $4/(1+x^2)$ at that point, and add it to the sum. After the loop, the sum multiplied by the step width gives us Pi.

Source Code

```
  GNU nano 7.2                          1_serial_pi.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000; // number of rectangles, more = more accurate
double step;

int main()
{
    double x, sum = 0.0, pi;
    double start_time, end_time;

    step = 1.0 / num_steps; // width of each rectangle

    start_time = omp_get_wtime(); // start timer

    int i;
    for (i = 0; i < num_steps; i++)
    {
        x = (i + 0.5) * step;          // midpoint of rectangle
        sum += 4.0 / (1.0 + x * x);    // height of curve at midpoint
    }

    pi = sum * step; // multiply sum by width to get area = Pi

    end_time = omp_get_wtime(); // stop timer

    printf("Pi = %.10f\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);

    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./1_serial_pi
Pi = 3.1415926536
Time = 0.1939 seconds
```

Conclusion

The serial version gives a correct result but uses only one thread.

Problem 2

Problem Title: Parallel PI

Lecture Number: Lec 2, Unit 2, Mod 3

Problem Description

Parallelize the serial pi program by splitting the loop across multiple threads using OpenMP. But no synchronization is added which introduces race conditions.

Parallelization Approach

Using `#pragma omp parallel for` to distribute the loop iterations across threads.

Solution Explanation

When multiple threads read and update `sum` at the same time, they interfere with each other, and updates get lost and the results are wrong. This is called a race condition.

Source Code

```
  GNU nano 7.2                                2_parallel_pi_race.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000;
double step;

int main()
{
    double x, sum = 0.0, pi;
    double start_time, end_time;

    step = 1.0 / num_steps;

    omp_set_num_threads(4);

    start_time = omp_get_wtime();

    int i;
    // all 4 threads run this loop and all write to the same 'sum' at the same time
    // all thread reads sum and all add their value and write back
    // one write overwrites the other --> lost updates --> wrong answer
    #pragma omp parallel for
    for (i = 0; i < num_steps; i++)
    {
        x = (i + 0.5) * step;
        sum += 4.0 / (1.0 + x * x); // RACE CONDITION HERE -- no protection on sum
    }

    pi = sum * step;

    end_time = omp_get_wtime();

    printf("Pi = %.10f  (expected ~3.1415926536)\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./2_parallel_pi_race
Pi = 1.2167015245  (expected ~3.1415926536)
Time = 0.2932 seconds
jannah@Jannah-PC:~/parallel_tasks$ ./2_parallel_pi_race
Pi = 1.0998384407  (expected ~3.1415926536)
Time = 0.3071 seconds
jannah@Jannah-PC:~/parallel_tasks$ ./2_parallel_pi_race
Pi = 1.2391554501  (expected ~3.1415926536)
Time = 0.2961 seconds
```

Output shows an incorrect value of Pi that is different each run because of race condition.

Conclusion

Parallelism without synchronization leads to incorrect results. This problem motivates all the solutions that follow.

Problem 3

Problem Title: Parallel PI – Cyclic Distribution

Lecture Number: Lec 2, Unit 2, Disc 2

Problem Description

Manually distribute loop iterations in a round-robin (cyclic) pattern based on thread IDs to avoid race conditions.

Parallelization Approach

Using #pragma omp parallel, each thread determines its ID via omp_get_thread_num() and computes only its assigned steps (e.g., Thread 0: 0, 4, 8...) and then stores its result in its own slot in a shared array, then the array is summed up serially.

Solution Explanation

The result is correct because each thread writes to its own array slot. However, performance may be poor due to false sharing which happens when threads write to different variables that happen to sit on the same cache line and the CPU treats the whole cache line as modified and forces other threads to reload it.

Source Code

```
  GNU nano 7.2                        3_parallel_pi_cyclic.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000;
double step;

int main()
{
    int nthreads;
    double pi;
    double sum[4]; // one slot per thread
    double start_time, end_time;

    step = 1.0 / num_steps;

    omp_set_num_threads(4);

    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        double x;
        int id = omp_get_thread_num();      // this thread's id (0,1,2,3)
        int nthrds = omp_get_num_threads(); // actual number of threads created

        if (id == 0) nthreads = nthrds; // save actual thread count from master thread

        int i;
        // cyclic distribution: thread 0 does steps 0,4,8,... thread 1 does 1,5,9,...
        for (i = id, sum[id] = 0.0; i < num_steps; i += nthrds)
        {
            x = (i + 0.5) * step;
            sum[id] += 4.0 / (1.0 + x * x); // each thread writes to its own slot
            // but sum[0] and sum[1] are neighbors in memory --> same cache line
        }
    }

    int i;
    for (i = 0, pi = 0.0; i < nthreads; i++)
        pi += sum[i] * step; // serial merge of partial sums

    end_time = omp_get_wtime();

    printf("Pi = %.10f\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./3_parallel_pi_cyclic
Pi = 3.1415926536
Time = 1.5300 seconds
```

Output shows correct Pi value. Timing is slow due to false sharing.

Conclusion

Cyclic distribution gives correct results but suffers from false sharing, which hurts performance. The next solution addresses this.

Problem 4

Problem Title: Parallel PI – Padded Array

Lecture Number: Lec 2, Unit 2, Disc 2

Problem Description

This version fixes the false sharing problem from the cyclic distribution by adding padding between each thread's slot in the array.

Parallelization Approach

Same cyclic approach but threads write to index thread_id * PADDING.

Solution Explanation

Each thread writes to index `thread_id * PADDING` in the array instead of just `thread_id`, spacing out elements (typically by a factor of 8 doubles / 64 bytes) to ensure each thread's data lands on a different cache line, eliminating false sharing.

Source Code

```
  GNU nano 7.2                                4_parallel_pi_padded.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000;
double step;

#define PAD 8 // 64-byte cache line / 8-byte double = 8 --> one slot per cache line

int main()
{
    int nthreads;
    double pi;
    double sum[4][PAD]; // sum[id][0] is the value, the rest is padding to push next slot to next cache line
    double start_time, end_time;

    step = 1.0 / num_steps;

    omp_set_num_threads(4);

    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        double x;
        int id = omp_get_thread_num();
        int nthrds = omp_get_num_threads();

        if (id == 0) nthreads = nthrds;

        int i;
        // same cyclic distribution as before
        for (i = id, sum[id][0] = 0.0; i < num_steps; i += nthrds)
        {
            x = (i + 0.5) * step;
            sum[id][0] += 4.0 / (1.0 + x * x);
            // now sum[0][0] and sum[1][0] are 8 doubles = 64 bytes apart
            // they are on different cache lines --> no false sharing
        }
    }

    int i;
    for (i = 0, pi = 0.0; i < nthreads; i++)
        pi += sum[i][0] * step;

    end_time = omp_get_wtime();

    printf("Pi = %.10f\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./4_parallel_pi_padded
Pi = 3.1415926536
Time = 0.6444 seconds
```

Output shows correct Pi value with improved timing compared to the non-padded version.

Conclusion

Padding is a simple technique to eliminate false sharing, but it requires knowledge of the cache line size of the target machine.

Problem 5

Problem Title: Parallel PI – Critical Section

Lecture Number: Lec 2, Unit 2, Disc 3

Problem Description

This version solves the race condition from Problem 2 by using a critical section to protect the shared sum variable.

Parallelization Approach

Using `#pragma omp parallel for` with a `#pragma omp critical` block around the update to `sum`. Only one thread is allowed inside the critical section at a time.

Solution Explanation

Each thread computes its partial sum to Pi and then enters the critical section to safely add it to the global sum. This prevents race conditions because no two threads can update `sum` at the same time. But the issue is that the threads have to wait for each other at the critical section, which reduces parallelism.

Source Code

```
 GNU nano 7.2                                5_parallel_pi_critical.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000;
double step;

int main()
{
    double sum = 0.0, pi;
    double start_time, end_time;

    step = 1.0 / num_steps;

    omp_set_num_threads(4);

    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        double x, local_sum = 0.0; // each thread has its own local_sum, no sharing
        int id = omp_get_thread_num();
        int nthrds = omp_get_num_threads();

        int i;
        // each thread computes its own portion using cyclic distribution
        for (i = id; i < num_steps; i += nthrds)
        {
            x = (i + 0.5) * step;
            local_sum += 4.0 / (1.0 + x * x); // safe -- only one thread touches local_sum
        }

        // merge into global sum, only one thread allowed here at a time
        #pragma omp critical
        {
            sum += local_sum; // serialized by critical section
        }
    }

    pi = sum * step;

    end_time = omp_get_wtime();

    printf("Pi = %.10f\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./5_parallel_pi_critical
Pi = 3.1415926536
Time = 0.0815 seconds
```

Output shows correct Pi value. Timing is slower than reduction (later solution) due to the overhead of serializing updates.

Conclusion

Critical sections are a simple and safe way to protect shared data, but they come with a performance cost because they serialize part of the work.

Problem 6

Problem Title: Parallel PI – Atomic

Lecture Number: Lec 2, Unit 2, Disc 3

Problem Description

This version uses `#pragma omp atomic` as a lighter alternative to the critical section.

Parallelization Approach

Same structure as the critical section version, but we replace `#pragma omp critical` with `#pragma omp atomic` on the update line.

Solution Explanation

Atomic operations are hardware-level instructions that perform a read-modify-write in a single step. They only work for simple operations like `sum += value and not` complex multi-line code blocks.

Source Code

```
 GNU nano 7.2                                  6_parallel_pi_atomic.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000;
double step;

int main()
{
    double sum = 0.0, pi;
    double start_time, end_time;

    step = 1.0 / num_steps;

    omp_set_num_threads(4);

    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        double x, local_sum = 0.0; // private to each thread
        int id = omp_get_thread_num();
        int nthrds = omp_get_num_threads();

        int i;
        for (i = id; i < num_steps; i += nthrds)
        {
            x = (i + 0.5) * step;
            local_sum += 4.0 / (1.0 + x * x);
        }

        // atomic protects a single read-modify-write operation at the hardware level
        // faster than critical but only works for simple operations like +=
        #pragma omp atomic
        sum += local_sum;
    }

    pi = sum * step;

    end_time = omp_get_wtime();

    printf("Pi = %.10f\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./6_parallel_pi_atomic
Pi = 3.1415926536
Time = 0.0773 seconds
```

Output shows correct Pi value with similar timing to the critical section version.

Conclusion

Atomic is preferred over critical when the protected operation is a single simple update. For anything more complex, critical sections are needed.

Problem 7

Problem Title: Parallel PI – Locks

Section Number: Section 9

Problem Description

This version uses OpenMP locks (`omp_lock_t`) to manually control access to the shared sum, as a lower-level alternative to critical sections.

Parallelization Approach

Initialize a lock with `omp_init_lock`, use `omp_set_lock` and `omp_unset_lock` around the sum update inside the parallel region, and destroy it after with `omp_destroy_lock`.

Solution Explanation

Locks give direct control over synchronization. Like a critical section only one thread can hold the lock at a time.

Source Code

```
  GNU nano 7.2                          7_parallel_pi_locks.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000;
double step;

int main()
{
    double sum = 0.0, pi;
    double start_time, end_time;
    omp_lock_t lock; // declare the lock

    step = 1.0 / num_steps;

    omp_init_lock(&lock); // initialize the lock before use

    omp_set_num_threads(4);

    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        double x, local_sum = 0.0;
        int id = omp_get_thread_num();
        int nthrds = omp_get_num_threads();

        int i;
        for (i = id; i < num_steps; i += nthrds)
        {
            x = (i + 0.5) * step;
            local_sum += 4.0 / (1.0 + x * x);
        }

        omp_set_lock(&lock);   // acquire the lock, other threads block here until released
        sum += local_sum;      // safe, only one thread in here at a time
        omp_unset_lock(&lock); // release the lock so next thread can enter
    }

    pi = sum * step;

    end_time = omp_get_wtime();

    omp_destroy_lock(&lock); // clean up the lock after we are done

    printf("Pi = %.10f\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./7_parallel_pi_locks
Pi = 3.1415926536
Time = 0.0760 seconds
```

Output shows correct Pi value. Timing is similar to the critical and atomic versions.

Conclusion

Locks are a lower-level synchronization tool similar to critical sections but more complex to use.

Problem 8

Problem Title: Parallel PI – Parallel for with Reduction

Lecture Number: Lec 2, Unit 2, Disc 4

Problem Description

This version uses OpenMP's built-in `reduction` clause, which is the cleanest solution to the race condition problem.

Parallelization Approach

We add `reduction(+:sum)` to the `#pragma omp parallel for` directive.

Solution Explanation

The reduction clause gives each thread its own private copy of `sum` initialized to 0, let each thread accumulate into its own copy, and then combine all copies using the + operator at the end of the parallel region.

Source Code

```
  GNU nano 7.2                          8_parallel_pi_reduction.c
#include <stdio.h>
#include "omp.h"

long long num_steps = 100000000;
double step;

int main()
{
    double x, sum = 0.0, pi;
    double start_time, end_time;

    step = 1.0 / num_steps;

    omp_set_num_threads(4);

    start_time = omp_get_wtime();

    // reduction(+:sum) handles everything automatically:
    // it gives each thread a private copy of sum initialized to 0
    // and then merges all copies using + at the end of the parallel region

    // no race condition, no manual arrays, no synchronization code needed
    #pragma omp parallel for private(x) reduction(+:sum)
    for (int i = 0; i < num_steps; i++)
    {
        x = (i + 0.5) * step;
        sum += 4.0 / (1.0 + x * x); // each thread writes to its own private copy of sum
    }                               // at the end all copies are added together automatically

    pi = sum * step;

    end_time = omp_get_wtime();

    printf("Pi = %.10f\n", pi);
    printf("Time = %.4f seconds\n", end_time - start_time);
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ ./8_parallel_pi_reduction
Pi = 3.1415926536
Time = 0.0627 seconds
```

Output shows correct Pi value with the best performance among all synchronized solutions because threads never wait for each other during the loop.

Conclusion

Reduction is the recommended way to parallelize accumulation operations in OpenMP. It is safe, fast, and requires minimal code changes from the serial version.

Problem 9

Problem Title: MPI Deadlock

Lecture Number: Lec3, MPI Processes and Messaging

Problem Description

When two processes both call `MPI_Send` before `MPI_Recv`, each block waiting for the other to receive first and neither proceeds, causing a deadlock.

Parallelization Approach

Two processes communicate using `MPI_Send` and `MPI_Recv` in the wrong order, both send first, both block, program hangs forever..

Solution Explanation

Process 0 calls `MPI_Send` to process 1, which blocks until process 1 is ready to receive. But process 1 is also stuck in its own `MPI_Send` to process 0. Neither proceeds to `MPI_Recv` — deadlock. The program must be killed manually.

Source Code

```
  GNU nano 7.2                          9_mpi_deadlock.c
#include <stdio.h>
#include "mpi.h"

#define MSG_SIZE 1000000 // large message
int main(int argc, char* argv[])
{
    int rank, size;
    int send_buf[MSG_SIZE], recv_buf[MSG_SIZE];
    MPI_Status status;

    MPI_Init(&argc, &argv);              // initialize MPI environment
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // get this process rank
    MPI_Comm_size(MPI_COMM_WORLD, &size); // get total number of processes

    // fill send buffer with this process's rank
    int i;
    for (i = 0; i < MSG_SIZE; i++) send_buf[i] = rank;

    if (rank == 0)
    {
        // process 0 tries to send first, blocks here waiting for process 1 to receive
        // but process 1 is also stuck trying to send -> deadlock
        MPI_Send(send_buf, MSG_SIZE, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(recv_buf, MSG_SIZE, MPI_INT, 1, 0, MPI_COMM_WORLD, &status);
        printf("Process 0 received first element: %d\n", recv_buf[0]); // never reached
    }
    else if (rank == 1)
    {
        // process 1 also tries to send first, blocks waiting for process 0 to receive
        // process 0 is not receiving yet, it is also stuck in MPI_Send
        MPI_Send(send_buf, MSG_SIZE, MPI_INT, 0, 0, MPI_COMM_WORLD);
        MPI_Recv(recv_buf, MSG_SIZE, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        printf("Process 1 received first element: %d\n", recv_buf[0]); // never reached
    }

    MPI_Finalize();
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ mpicc 9_mpi_deadlock.c -o 9_mpi_deadlock
jannah@Jannah-PC:~/parallel_tasks$ mpirun -n 2 ./9_mpi_deadlock
^Cjannah@Jannah-PC:~/parallel_tasks$ ^C
```

The program produced no output and was manually terminated with Ctrl+C, confirming deadlock.

Conclusion

Order of send and receive operations matters in MPI. When both sides send first, neither can progress.

Problem 10

Problem Title: MPI Deadlock Fix — MPI_Sendrecv

Lecture Number: Lec3, MPI Processes and Messaging

Problem Description

Fix the deadlock from Problem 9 using `MPI_Sendrecv`, which performs a send and receive simultaneously in a single call.

Parallelization Approach

Same two-process scenario but both sides use `MPI_Sendrecv` so the MPI runtime coordinates both operations together without blocking.

Solution Explanation

`MPI_Sendrecv` takes both the send and receive parameters in one call. The MPI runtime handles the coordination internally so neither side blocks waiting for the other. The result is correct and the program completes normally.

Source Code

```
  GNU nano 7.2                          10_mpi_sendrecv.c
#include <stdio.h>
#include "mpi.h"

#define MSG_SIZE 1000000

int main(int argc, char* argv[])
{
    int rank, size;
    static int send_buf[MSG_SIZE], recv_buf[MSG_SIZE];
    MPI_Status status;

    MPI_Init(&argc, &argv);               // initialize MPI environment
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // get this process rank
    MPI_Comm_size(MPI_COMM_WORLD, &size); // get total number of processes

    // fill send buffer with this process's rank
    int i;
    for (i = 0; i < MSG_SIZE; i++) send_buf[i] = rank;

    if (rank == 0)
    {
        // send and receive simultaneously -- no blocking, no deadlock
        MPI_Sendrecv(
            send_buf, MSG_SIZE, MPI_INT, 1, 0,   // send to process 1
            recv_buf, MSG_SIZE, MPI_INT, 1, 0,   // receive from process 1
            MPI_COMM_WORLD, &status
        );
        printf("Process 0 sent %d, received %d\n", send_buf[0], recv_buf[0]);
    }
    else if (rank == 1)
    {
        MPI_Sendrecv(
            send_buf, MSG_SIZE, MPI_INT, 0, 0,   // send to process 0
            recv_buf, MSG_SIZE, MPI_INT, 0, 0,   // receive from process 0
            MPI_COMM_WORLD, &status
        );
        printf("Process 1 sent %d, received %d\n", send_buf[0], recv_buf[0]);
    }

    MPI_Finalize();
    return 0;
}
```

Execution Results

```
jannah@Jannah-PC:~/parallel_tasks$ mpicc 10_mpi_sendrecv.c -o 10_mpi_sendrecv
jannah@Jannah-PC:~/parallel_tasks$ mpirun -n 2 ./10_mpi_sendrecv
Process 0 sent 0, received 1
Process 1 sent 1, received 0
```

Conclusion

Order of send and receive operations matters in MPI. When both sides send first, neither can progress.